

Oracle JSON & XML Handling in PL/SQL

Generating, querying & using JSON and XML inside procedures · Oracle 19c

This guide covers how Oracle stores, generates, and queries **JSON** and **XML**, and how to work with both inside **PL/SQL procedures** — parsing input payloads and building output documents. Examples target Oracle 19c (notes added where 21c differs).

Part 1 — JSON in Oracle

1.1 Storing JSON

In 19c, JSON lives in a **VARCHAR2**, **CLOB**, or **BLOB** column guarded by an **IS JSON** constraint. (A dedicated **JSON** data type arrived in 21c.)

```
CREATE TABLE cust_json (
  id    NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  doc   CLOB,
  CONSTRAINT ck_doc_json CHECK (doc IS JSON)
);

INSERT INTO cust_json (doc) VALUES (
  '{"cust_code":101,"name":"Ali","emails":["a@x.com","b@y.com"],
    "address":{"city":"Dubai","zip":"00000"}}'
);
```

1.2 Querying JSON

Function	Purpose	Example
JSON_VALUE	Extract one scalar value	JSON_VALUE(doc, '\$.name') → Ali
JSON_QUERY	Extract an object/array fragment	JSON_QUERY(doc, '\$.emails')
JSON_EXISTS	Test a path exists (WHERE)	JSON_EXISTS(doc, '\$.address.city')
JSON_TABLE	Turn JSON into rows/columns	see below
dot-notation	Simple access (alias + IS JSON)	c.doc.name

```
-- scalar extraction with datatype + default
SELECT JSON_VALUE(doc, '$.cust_code' RETURNING NUMBER)          AS cust_code,
       JSON_VALUE(doc, '$.address.city' DEFAULT 'N/A' ON ERROR) AS city
FROM   cust_json;

-- shred an array into rows with JSON_TABLE
SELECT jt.*
FROM   cust_json c,
       JSON_TABLE(c.doc, '$'
         COLUMNS (
           cust_code    NUMBER          PATH '$.cust_code',
           name         VARCHAR2(50)    PATH '$.name',
           city         VARCHAR2(50)    PATH '$.address.city',
           NESTED PATH '$.emails[*]'    COLUMNS (email VARCHAR2(100) PATH '$')
         )) jt;
```

1.3 Generating JSON

```
SELECT JSON_OBJECT(
  'cust_code' VALUE cust_code,
  'name'      VALUE cust_name,
  'orders'    VALUE JSON_ARRAYAGG(order_no)
) AS json_doc
FROM   customer JOIN orders USING (cust_code)
GROUP BY cust_code, cust_name;
```

Key generators: **JSON_OBJECT**, **JSON_ARRAY**, **JSON_OBJECTAGG** (rows→one object), **JSON_ARRAYAGG** (rows→array).

1.4 JSON in a procedure — parsing input (JSON_OBJECT_T API)

```
CREATE OR REPLACE PROCEDURE process_customer_json (p_json IN CLOB)
IS
    l_obj  JSON_OBJECT_T;
    l_arr  JSON_ARRAY_T;
    l_addr JSON_OBJECT_T;
    l_code NUMBER;
    l_name VARCHAR2(100);
    l_city VARCHAR2(100);
BEGIN
    l_obj := JSON_OBJECT_T.parse(p_json);           -- parse text into object
    l_code := l_obj.get_Number('cust_code');
    l_name := l_obj.get_String('name');

    IF l_obj.has('address') THEN
        l_addr := l_obj.get_Object('address');
        l_city := l_addr.get_String('city');
    END IF;

    l_arr := l_obj.get_Array('emails');
    FOR i IN 0 .. l_arr.get_size - 1 LOOP          -- 0-based!
        DBMS_OUTPUT.put_line('Email: ' || l_arr.get_String(i));
    END LOOP;

    DBMS_OUTPUT.put_line(l_code || ' / ' || l_name || ' / ' || l_city);
    -- ... INSERT/UPDATE using extracted values ...
END;
/
```

1.5 JSON in a procedure — building output

```
CREATE OR REPLACE PROCEDURE build_customer_json (
    p_cust_code IN NUMBER,
    p_json      OUT CLOB)
IS
    l_obj JSON_OBJECT_T := JSON_OBJECT_T();
    l_arr JSON_ARRAY_T  := JSON_ARRAY_T();
BEGIN
    l_obj.put('cust_code', p_cust_code);
    l_obj.put('name', 'Ali');
    l_obj.put('active', TRUE);
    l_arr.append('a@x.com');
    l_arr.append('b@y.com');
    l_obj.put('emails', l_arr);
    p_json := l_obj.to_Clob;                     -- serialize back to text
END;
/
```

Part 2 — XML in Oracle

2.1 Storing XML (XMLTYPE)

```
CREATE TABLE cust_xml (
    id    NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    doc   XMLTYPE
);

INSERT INTO cust_xml (doc) VALUES (
    XMLTYPE('<customer code="101">
        <name>Ali</name>
        <emails><email>a@x.com</email><email>b@y.com</email></emails>
        <address><city>Dubai</city></address>
    </customer>'));

```

2.2 Querying XML

Function	Purpose	Example
<code>XMLQUERY</code>	XQuery, returns XML/scalar	<code>XMLQUERY('/customer/name/text()' PASSING doc RETURNING CONTENT)</code>
<code>XMLTABLE</code>	XML → rows/columns (preferred)	see below
<code>XMLEXISTS</code>	Test node exists (WHERE)	<code>XMLEXISTS('/customer/address' PASSING doc)</code>
<code>.getClobVal()</code>	Serialize XMLType to text	<code>doc.getClobVal()</code>
<code>EXTRACTVALUE</code> / <code>EXTRACT</code>	Legacy — deprecated	use XMLTABLE/XMLQUERY instead

```
-- shred XML into relational rows
SELECT x.cust_code, x.name, x.city, x.email
FROM   cust_xml c,
       XMLTABLE('/customer' PASSING c.doc
                COLUMNS
                  cust_code NUMBER      PATH '@code',
                  name      VARCHAR2(50) PATH 'name',
                  city      VARCHAR2(50) PATH 'address/city',
                  email     VARCHAR2(100) PATH 'emails/email[1]') x;

-- single scalar value
SELECT XMLCAST(XMLQUERY('/customer/name/text()' PASSING doc RETURNING CONTENT)
              AS VARCHAR2(50)) AS name
FROM   cust_xml;
```

2.3 Generating XML

```
SELECT XMLELEMENT("customer",
  XMLATTRIBUTES(cust_code AS "code"),
  XMLFOREST(cust_name AS "name"),
  XMLELEMENT("orders",
    XMLAGG(XMLELEMENT("order", order_no)))
).getClobVal() AS xml_doc
FROM   customer JOIN orders USING (cust_code)
GROUP BY cust_code, cust_name;
```

Key generators: `XMLELEMENT`, `XMLATTRIBUTES`, `XMLFOREST`, `XMLAGG`, `XMLCONCAT`.

2.4 XML in a procedure — parsing input

```
CREATE OR REPLACE PROCEDURE process_customer_xml (p_xml IN CLOB)
IS
  l_xml XMLTYPE := XMLTYPE(p_xml);
  l_code NUMBER;
  l_name VARCHAR2(100);
  l_city VARCHAR2(100);
BEGIN
  SELECT XMLCAST(XMLQUERY('/customer/@code' PASSING l_xml RETURNING CONTENT) AS NUMBER),
         XMLCAST(XMLQUERY('/customer/name/text()' PASSING l_xml RETURNING CONTENT) AS VARCHAR2(100)),
         XMLCAST(XMLQUERY('/customer/address/city/text()' PASSING l_xml RETURNING CONTENT) AS VARCHAR2(100))
  INTO   l_code, l_name, l_city
  FROM   dual;

  DBMS_OUTPUT.put_line(l_code || ' / ' || l_name || ' / ' || l_city);

  -- Loop repeating nodes with XMLTABLE
  FOR r IN (SELECT email
            FROM   XMLTABLE('/customer/emails/email' PASSING l_xml
                           COLUMNS email VARCHAR2(100) PATH '.')) LOOP
    DBMS_OUTPUT.put_line('Email: ' || r.email);
  END LOOP;
END;
/
```

2.5 XML in a procedure — building output

```
CREATE OR REPLACE PROCEDURE build_customer_xml (  
    p_cust_code IN NUMBER,  
    p_xml      OUT CLOB)  
IS  
    l_xml XMLTYPE;  
BEGIN  
    SELECT XMLELEMENT("customer",  
        XMLATTRIBUTES(p_cust_code AS "code"),  
        XMLFOREST('Ali' AS "name", 'Dubai' AS "city"))  
    INTO l_xml  
    FROM dual;  
    p_xml := l_xml.getClobVal();  
END;  
/
```

Part 3 — JSON vs XML: which to use

Aspect	JSON	XML
Best for	REST APIs, lightweight data, web	Documents, namespaces, XSD schemas
Storage	CLOB+IS JSON (19c), JSON type (21c)	XMLTYPE
Shred to rows	JSON_TABLE	XMLTABLE
PL/SQL API	JSON_OBJECT_T / JSON_ARRAY_T	XMLTYPE methods + XQuery
Indexing	function-based / JSON search index	XMLIndex
Verdict	Preferred for new/integration work	Use when source is XML or needs XSD validation

Common gotchas

- **JSON PL/SQL arrays are 0-based** (0 .. get_size-1); SQL is otherwise 1-based.
- Prefer **JSON_TABLE / XMLTABLE** over manual looping when shredding to relational data — far faster.
- **EXTRACTVALUE/EXTRACT** for XML are **deprecated** — use **XMLQUERY/XMLTABLE**.
- Always add **ON ERROR / DEFAULT ... ON ERROR** to **JSON_VALUE** in production so bad data doesn't raise.
- For large documents, use **CLOB/BLOB**, not **VARCHAR2**.